

EPI Monitor v6.0

- Python 3.10+
- YOLOv8
- OpenCV
- Firestore (opcional)

Sistema de visão computacional para detecção em tempo real de EPI (Equipamento de Proteção Individual) — **capacete** e **colete** — via webcam, utilizando modelos YOLOv8. Gera logs automáticos em JSON e TXT, exibe HUD com status por pessoa e suporta integração opcional com Firestore e GPU.

Arquitetura do Sistema

O sistema é organizado em 4 camadas:

Camada 1 — Entrada

Componente	Descrição
Webcam	Captura de frames em tempo real. Índice configurável via <code>CONFIG["camera_index"]</code> (padrão: 1).
yolov8n.pt	Modelo base para detecção de pessoas (baixado automaticamente pelo Ultralytics).
best.pt	Modelo custom de EPI (capacete/colete). Deve ser fornecido pelo usuário.
CONFIG	Dicionário central com todos os parâmetros: device, confiança, classes, caminhos, etc.

Camada 2 — EPIMonitor (loop principal)

Classe	Responsabilidade
EPIDetector	Executa inferência YOLO. Suporta multi-rotação (0°/90°/270°) para câmeras inclinadas. Aplica filtro anti-falso-positivo e associa EPI detectado a cada pessoa por sobreposição de bounding box.
PersonTracker	Mantém IDs estáveis por pessoa entre frames usando matching por IoU. Evita duplicação de logs. Parâmetro <code>max_lost = 15 frames</code> .
EPILogger	Registra eventos por pessoa (ID, capacete, colete, timestamp). Salva JSON a cada 30s e TXT ao encerrar. Usa <code>NumpyEncoder</code> para serialização segura.
HUDRenderer	Renderiza bounding boxes coloridas (verde/laranja/vermelho), banners de violação, painel de log lateral, ícones de status e FPS em tempo real.

Camada 3 — Saída

Saída	Descrição
Janela OpenCV	

	Exibe o vídeo com HUD em tempo real. Tecla <code>Q</code> para encerrar.
<code>logs/epi_log_*.json</code>	Log estruturado com todas as detecções, salvo automaticamente a cada 30 segundos.
<code>logs/epi_summary_*.txt</code>	Resumo textual com estatísticas (total de pessoas, violações, taxa de conformidade) gerado ao encerrar.
Terminal (stdout)	Mensagens de status, erros e avisos em tempo real.

■ Camada 4 — Opcional (datasetproprioiv2.py)

Componente	Descrição
<code>FirebaseManager</code>	Integração assíncrona com Firebase Firestore para persistência de logs na nuvem.
<code>AccessController</code>	Libera ou bloqueia acesso com base no resultado da detecção de EPI.
RFID Simulado	Tecla <code>R</code> abre input para identificação do funcionário (substitui Arduino/serial).
GPU / FP16	Suporte a CUDA com inferência em meia precisão para maior velocidade.

Estrutura de Arquivos

```
projeto/
├── epi_detector_v6.py      # Script principal (versão simples)
├── datasetproprioiv2.py   # Versão completa (Firebase + GPU + RFID)
├── requirements.txt       # Dependências base
├── best.pt                # Modelo EPI custom (você fornece)
├── datasetproprioiv2.pt   # Modelo custom alternativo (opcional)
├── firebase_key.json      # Chave Firebase (opcional)
└── logs/                  # Criado automaticamente
    ├── epi_log_*.json
    └── epi_summary_*.txt
```

Requisitos

- Windows 10/11 (ou Linux/macOS)
- Python 3.10 ou superior (testado em 3.12)
- Webcam conectada
- Modelo de EPI treinado (`best.pt`)
- GPU NVIDIA com CUDA (opcional, para melhor FPS)
- Conta Firebase (opcional, para Firestore)

Instalação

1. Clonar / baixar o projeto

```
git clone https://github.com/seu-usuario/epi-monitor.git
cd epi-monitor
```

2. Criar ambiente virtual

```
# Windows
python -m venv .venv
.\.venv\Scripts\activate

# Linux / macOS
python3 -m venv .venv
source .venv/bin/activate
```

3. Atualizar pip e instalar dependências base

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Conteúdo do requirements.txt:

`opencv-python>=4.9.0` | `ultralytics>=8.2.0` | `numpy>=1.26.0`

O pacote `ultralytics` instala o PyTorch automaticamente (versão CPU por padrão).

4. (Opcional) Instalar dependências extras — Firebase e GPU

```
# Firebase Firestore
pip install firebase-admin

# PyTorch com suporte CUDA (GPU NVIDIA) - substitua cu121 pela sua versão CUDA
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
```

Verificar versão CUDA: Execute `nvidia-smi` no terminal para ver a versão CUDA instalada e escolher o build correto do PyTorch em pytorch.org/get-started.

5. Colocar os modelos na pasta do projeto

- `best.pt` — modelo de EPI para o `epi_detector_v6.py`
- `datasetproprio2.pt` — modelo alternativo para o `datasetproprio2.py`
- `firebase_key.json` — chave de serviço do Firebase (se usar Firestore)

Como Executar

Opcao A — epi_detector_v6.py (recomendado para comecar)

```
python epi_detector_v6.py
```

- Pressione `Q` para encerrar
- Logs salvos automaticamente em `logs/`

Opcao B — datasetproprioiv2.py (Firebase + GPU + RFID)

```
python datasetproprioiv2.py
```

- Pressione `R` para simular leitura de RFID
- Pressione `Q` para encerrar

Configuracoes Importantes

Parametro (CONFIG)	Arquivo	Descricao
<code>camera_index</code>	v6 / v2	Indice da webcam (0 = integrada, 1 = externa)
<code>ppe_model</code>	v6	Caminho para o modelo de EPI (ex: <code>best.pt</code>)
<code>model_path</code>	v2	Caminho para o modelo custom (<code>datasetproprioiv2.pt</code>)
<code>helmet_classes</code>	v6 / v2	Nomes das classes de capacete no seu modelo
<code>vest_classes</code>	v6 / v2	Nomes das classes de colete no seu modelo
<code>device</code>	v6 / v2	<code>auto</code> , <code>cpu</code> , <code>cuda</code> ou <code>mps</code>
<code>multi_rotation</code>	v6	<code>True</code> melhora deteccao em cameras inclinadas (custa FPS)
<code>half</code>	v2	<code>True</code> ativa FP16 (somente GPU)
<code>firebase_key</code>	v2	Caminho para <code>firebase_key.json</code>
<code>firebase_col</code>	v2	Nome da colecao no Firestore

Solucao de Problemas

Erro	Causa	Solucao
Camera nao abre	Indice errado	Troque <code>camera_index</code> para 0 ou 1
<code>FileNotFoundError:</code> <code>best.pt</code>	Modelo ausente	Coloque o <code>.pt</code> na pasta do projeto ou ajuste o caminho no CONFIG

Classes nao detectadas	Nomes errados	Edite <code>helmet_classes</code> e <code>vest_classes</code> para bater com as classes do seu modelo
<code>ModuleNotFoundError: serial</code>	pyserial ausente	<code>pip install pyserial</code> (ou use a versao sem Arduino)
FPS baixo	CPU / resolucao alta	Defina <code>device: cuda</code> , reduza <code>infer_size</code> ou desative <code>multi_rotation</code>
<code>JSON serializable int32</code>	Tipo NumPy no log	Ja corrigido na v6 com <code>NumpyEncoder</code>
Firebase nao conecta	Chave invalida	Verifique o caminho de <code>firebase_key.json</code> e as permissoes do Firestore

Teclas de Atalho

Tecla	Acao
<code>Q</code>	Encerrar o sistema e salvar o resumo TXT
<code>R</code>	Simular leitura de RFID (somente datasetproprio2.py)

Dependencias Resumidas

```
# Base (obrigatorio)
opencv-python>=4.9.0
ultralytics>=8.2.0
numpy>=1.26.0

# Extras (opcional)
firebase-admin      # Firestore
torch torchvision  # GPU/CUDA
pyserial            # Arduino (nao necessario na versao simulada)
```

Dica: Para verificar se a GPU esta sendo usada, observe o terminal ao iniciar — o sistema imprime o device selecionado (`cuda:0` ou `cpu`).

Licenca

Defina aqui a licenca do projeto (ex: MIT, Apache-2.0). Consulte choosealicense.com para escolher a mais adequada.